

Keycloak

The version of keycloak need to be: 20.0.5. For SO to work

Docker

```
docker run -p 8081:8080 -e KEYCLOAK_ADMIN=admin -e  
KEYCLOAK_ADMIN_PASSWORD=admin quay.io/keycloak/keycloak:20.0.2 start-dev
```

new docker

```
docker run -p 8081:8080 -e KEYCLOAK_ADMIN=admin -e  
KEYCLOAK_ADMIN_PASSWORD=admin quay.io/keycloak/keycloak:26.5.5 start-dev
```

Clients

Системта работи с два клиента. Първия е за REST API а второто от замото [SO](#).

Клиенте са описани като JSON файлове намиращи се в пакета web/api. JSON - ите носят имената на REALM-а а в тях са описани кои са клиентите.

Set up Wildfly config file

Directory structure

```
{wildfly-home}\standalone\configuration\keycloak-configurations  
|  
|-keycloak.properties  
|-web  
|   |-Realm-name.json  
|-api  
|   |-Realm-name.json
```

keycloak.properties

```
sso.url=http://localhost:8081/  
sso.user=admin  
sso.password=admin  
sso.default-realm=SmartOffice-Local  
api.default-realm=SmartOffice-Local
```

API Realm

Realm-name.json

```
{
  "realm": "Realm-name",
  "auth-server-url": "http://localhost:8081/", //where you host the container
  "ssl-required": "external",
  "resource": "so-api",
  "public-client": false,
  "credentials": {
    "secret": "{ClientSecret}"
  },
  "enable-basic-auth": true
}
```

Web Realm

Realm-name.json

```
{
  "realm": "Realm-name",
  "auth-server-url": "http://localhost:8081/", //where you host the container
  "ssl-required": "external",
  "resource": "so-api",
  "credentials": {
    "secret": "{ClientSecret}"
  },
  "confidential-port": 0
}
```

Set up web client

General Settings

Client ID * ⓘ

so-web

Name ⓘ

Description ⓘ

Always display in console ⓘ

Off

Access settings

Root URL ⓘ

Home URL ⓘ

Valid redirect URIs ⓘ

*

+

Add valid redirect URIs

Valid post logout redirect URIs ⓘ

+

Add valid post logout redirect URIs

Web origins ⓘ

*

+

Add web origins

Admin URL ⓘ

Capability config

Client authentication ⓘ ☒ On

Authorization ⓘ ☒ On

Authentication flow ☒ Standard flow ⓘ

☒ Direct access grants ⓘ

☒ Implicit flow ⓘ

☒ Service accounts roles ⓘ

☐ OAuth 2.0 Device Authorization Grant ⓘ

☐ OIDC CIBA Grant ⓘ

Login settings

Login theme ⓘ

Choose...

Consent required ⓘ ☐ Off

Display client on screen ⓘ ☐ Off

Client consent screen text ⓘ

Logout settings

Front channel logout ⓘ ☒ On

Front-channel logout URL ⓘ

Backchannel logout URL ⓘ

Backchannel logout session required ⓘ ☒ On

Backchannel logout revoke offline sessions ⓘ ☐ Off

Creating group

1- create group

name -> SO-All-Superusers

! important the name of the group NEEDS TO BE 'SO-All-Superusers'. Do not make one up.

2- add mapping to so-web
client-> so-web->client scopes -> so-web-decocoated -> add predefined mappers -> groups ->add

Give it a name of 'groups' and token claim name 'groups'

Add predefined mappers

Choose any of the predefined mappings from this table

Q Search for mapper

→

11 - 20 ▾ < >

☐	Name	Description
☐	gender	Map a custom user attribute to a token claim.
☐	given name	Map a built in user property (email, firstName, lastName) to a token claim.
☒	groups	Map a user realm role to a token claim.
☐	gss delegation credential	Map a custom user session note to a token claim.
☐	Impersonator User ID	Map a custom user session note to a token claim.
☐	Impersonator Username	Map a custom user session note to a token claim.
☐	locale	Map a custom user attribute to a token claim.
☐	middle name	Map a custom user attribute to a token claim.
☐	nickname	Map a custom user attribute to a token claim.
☐	phone number	Map a custom user attribute to a token claim.

11 - 20 ▾ < >

Add

Cancel

Add mapper

If you want more fine-grain control, you can create protocol mapper on this client

Mapper type	Group Membership
Name * ?	groups
Token Claim Name ?	groups
Full group path ?	☐ Off
Add to ID token ?	☒ On
Add to access token ?	☒ On
Add to userinfo ?	☒ On
Save Cancel	

"confidential-port": 443,

ivaylo.rumenov@unicard-uk.com

!Ivo2000plamjojo1